

Index Mapping (or Trivial Hashing) with negatives allowed

Last Updated : 02 Jun, 2023



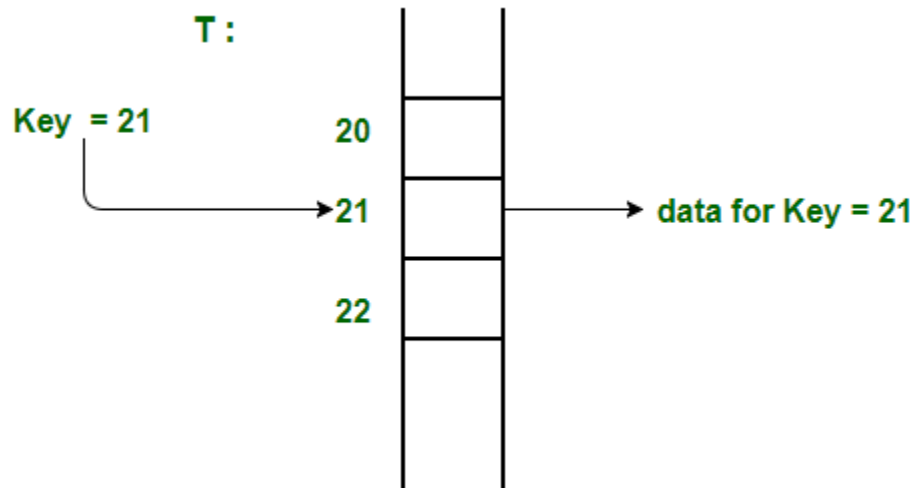
Index Mapping (also known as Trivial Hashing) is a simple form of hashing where the data is directly mapped to an index in a hash table. The hash function used in this method is typically the identity function, which maps the input data to itself. In this case, the key of the data is used as the index in the hash table, and the value is stored at that index.

For example, if we have a hash table of size 10 and we want to store the value "apple" with the key "a", the trivial hashing function would simply map the key "a" to the index "a" in the hash table, and store the value "apple" at that index.

One of the main advantages of Index Mapping is its simplicity. The hash function is easy to understand and implement, and the data can be easily retrieved using the key. However, it also has some limitations. The main disadvantage is that it can only be used for small data sets, as the size of the hash table has to be the same as the number of keys. Additionally, it doesn't handle collisions, so if two keys map to the same index, one of the data will be overwritten.

Given a limited range array contains both positive and non-positive numbers, i.e., elements are in the range from -MAX to +MAX. Our task is to search if some number is present in the array or not in $O(1)$ time.

Since the range is limited, we can use [index mapping](#) (or trivial hashing). We use values as the index in a big array. Therefore we can search and insert elements in $O(1)$ time.



How to handle negative numbers?

The idea is to use a 2D array of size `hash[MAX+1][2]`

Algorithm:

Assign all the values of the hash matrix as 0.

Traverse the given array:

- *If the element `ele` is non negative assign
 - `hash[ele][0]` as 1.*
- *Else take the absolute value of `ele` and
 - assign `hash[ele][1]` as 1.*

To search any element `x` in the array.

- If `X` is non-negative check if `hash[X][0]` is 1 or not. If `hash[X][0]` is one then the number is present else not present.

- If X is negative take the absolute value of X and then check if hash[X][1] is 1 or not. If hash[X][1] is one then the number is present

Below is the implementation of the above idea.

C++

Java

Python3

C#

JavaScript

```
// CPP program to implement direct index mapping
// with negative values allowed.
#include <bits/stdc++.h>
using namespace std;
#define MAX 1000

// Since array is global, it is initialized as 0.
bool has[MAX + 1][2];

// searching if X is Present in the given array
// or not.
bool search(int X)
{
    if (X >= 0) {
        if (has[X][0] == 1)
            return true;
        else
            return false;
    }

    // if X is negative take the absolute
    // value of X.
    X = abs(X);
    if (has[X][1] == 1)
        return true;

    return false;
}

void insert(int a[], int n)
{
    for (int i = 0; i < n; i++) {
        if (a[i] >= 0)
            has[a[i]][0] = 1;
    }
}
```



```

        else
            has[abs(a[i])][1] = 1;
    }
}

// Driver code
int main()
{
    int a[] = { -1, 9, -5, -8, -5, -2 };
    int n = sizeof(a)/sizeof(a[0]);
    insert(a, n);
    int X = -5;
    if (search(X) == true)
        cout << "Present";
    else
        cout << "Not Present";
    return 0;
}

```

Output

Present

Time Complexity: The time complexity of the above algorithm is $O(N)$, where N is the size of the given array.

Space Complexity: The space complexity of the above algorithm is $O(N)$, because we are using an array of max size.

 Comment

More info 

Advertise with us

Next Article 

Separate Chaining Collision Handling
Technique in Hashing

Similar Reads

Hashing in Data Structure

Hashing is a technique used in data structures that efficiently stores and retrieves data in a way that allows for quick access. Hashing involves mapping data to a specific index in a hash table (an array of...

🕒 3 min read

Introduction to Hashing

Hashing refers to the process of generating a small sized output (that can be used as index in a table) from an input of typically large and variable size. Hashing uses mathematical formulas known as hash...

🕒 7 min read

What is Hashing?

Hashing refers to the process of generating a fixed-size output from an input of variable size using the mathematical formulas known as hash functions. This technique determines an index or location for the...

🕒 3 min read

Index Mapping (or Trivial Hashing) with negatives allowed

Index Mapping (also known as Trivial Hashing) is a simple form of hashing where the data is directly mapped to an index in a hash table. The hash function used in this method is typically the identity...

🕒 7 min read

Separate Chaining Collision Handling Technique in Hashing

Separate Chaining is a collision handling technique. Separate chaining is one of the most popular and commonly used techniques in order to handle collisions. In this article, we will discuss about what is...

🕒 3 min read

Open Addressing Collision Handling technique in Hashing

Open Addressing is a method for handling collisions. In Open Addressing, all elements are stored in the hash table itself. So at any point, the size of the table must be greater than or equal to the total number...

🕒 7 min read

Double Hashing

Double hashing is a collision resolution technique used in hash tables. It works by using two hash functions to compute two different hash values for a given key. The first hash function is used to comput...

🕒 15+ min read

Load Factor and Rehashing

Prerequisites: Hashing Introduction and Collision handling by separate chaining How hashing works: For insertion of a key(K) - value(V) pair into a hash map, 2 steps are required: K is converted into a small...

🕒 15+ min read

Easy problems on Hashing



Intermediate problems on Hashing

